

Solution Design and Architecture

1. Problem

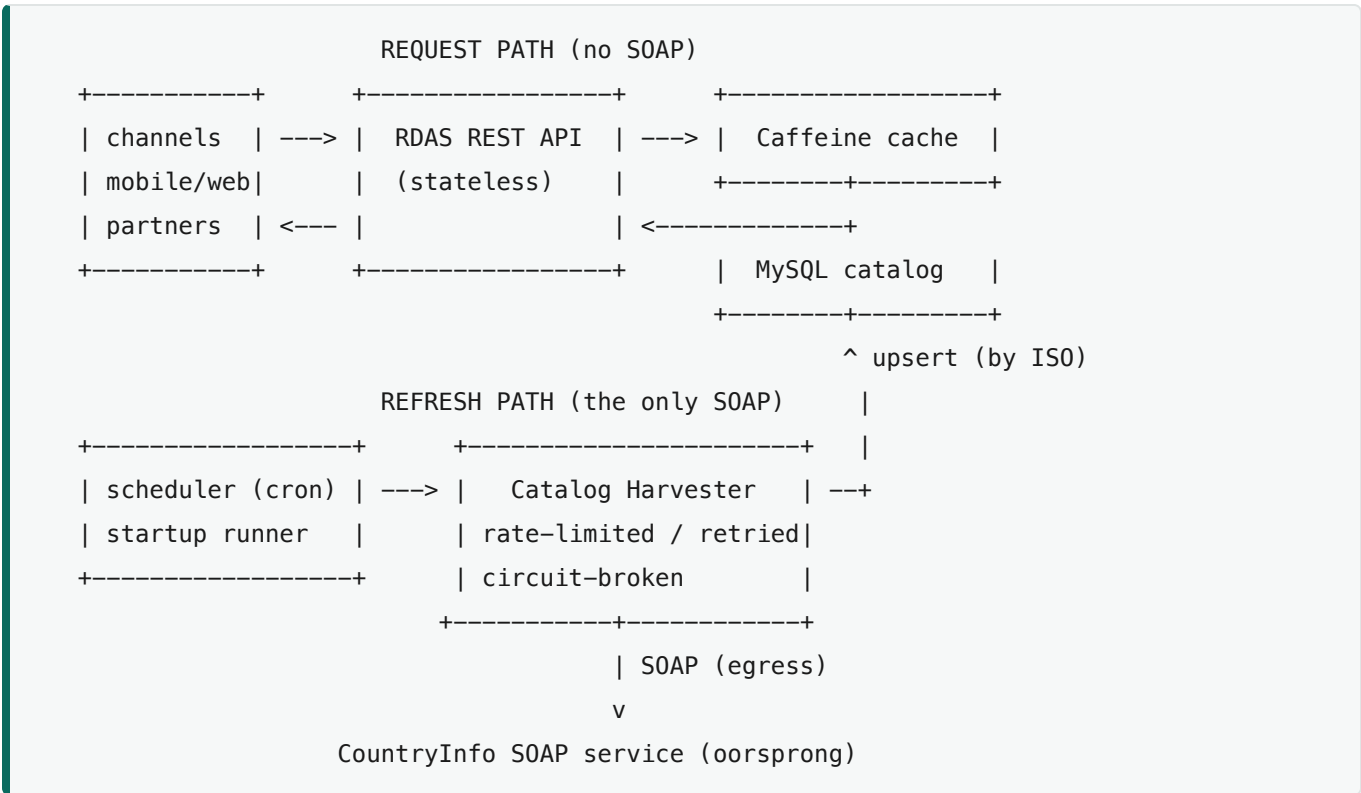
Multiple channels (mobile, web, partner APIs, internal ops) need country, currency, language and geographical reference data. Today each channel calls the third-party CountryInfo SOAP service directly, which causes inconsistent responses, poor performance from repeated SOAP calls, no filtering/pagination, no auditability, no shared caching, and SOAP credentials spread across many applications.

RDAS replaces that with one service that owns the SOAP integration and exposes clean REST/JSON to everyone.

2. Key architectural choice: a materialized catalog

The SOAP API exposes only narrow, single-record operations and is rate-limited. It cannot search, filter, paginate or sort. Calling it on the request path would be slow and fragile.

RDAS instead **materializes a complete local catalog** of all countries and the reference lists, and serves every read from that catalog. SOAP is consumed only by a background harvester.



3. Components

Component	Responsibility
CountryInfoSoapClient	The only SOAP caller. Wraps each operation with a rate limiter (under 100/min), retry, circuit breaker and fallback.
CatalogHarvestService	Builds/refreshes the catalog: 4 list calls + one FullCountryInfo per country, paced and upserted by ISO code. Scheduled daily and on startup if empty/stale. Single-flight.
CountryQueryService	Read side. Specification-based filtering, pagination, sorting; Caffeine-cached; maps entities to DTOs. Never calls SOAP.
Country , ReferenceItem	JPA entities of the catalog (countries + continent/currency/language reference lists).
Controllers	CountryController , ReferenceController , AdminController .
CatalogHealthIndicator	Surfaces catalog size/freshness on /actuator/health (not in the readiness group).

4. Data model

```
Country (table: country)
  id (internal PK)      isoCode (unique, public id)  name
  capitalCity  phoneCode  continentCode  continentName
  currencyCode  currencyName  flagUrl
  languages [CountryLanguage{code,name}] (table: country_language)
  createdAt  updatedAt  sourceRefreshedAt

ReferenceItem (table: reference_item)  -- continents, currencies, languages
  id  type (CONTINENT|CURRENCY|LANGUAGE)  code  name  unique(type,code)
```

Indexes on `iso_code` (unique), `name`, `continent_code`, `currency_code` support the filter and sort paths.

5. How requirements map to the design

Business need	Mechanism
Search by country name	<code>name</code> filter, partial case-insensitive <code>LIKE</code> .
Filter by continent / currency / language	JPA Specifications (continent and language match code or name).
Retrieve country details	<code>GET /countries/{isoCode}</code> .
Countries sharing a currency	<code>GET /countries/{isoCode}/currency-peers</code> .
Pagination + sorting	Spring Data <code>Pageable</code> , whitelisted sort fields, <code>PagedResponse</code> envelope.
Reduce SOAP traffic / caching	Materialized catalog + Caffeine; reads never call SOAP.
Auditability	<code>createdAt</code> / <code>updatedAt</code> / <code>sourceRefreshedAt</code> ; harvest status and structured logs.
One place holding SOAP credentials	Only RDAS integrates SOAP; channels never see it.

6. Cross-cutting concerns

- **Resilience:** Resilience4j (rate limiter, retry, circuit breaker, fallback); catalog persistence means a SOAP outage does not affect reads.
- **Observability:** Micrometer + Prometheus, Actuator health groups (readiness excludes SOAP), structured JSON logs under the `prod` profile.
- **Security boundary:** reference data is public, so ISO codes are exposed directly; admin endpoints are flagged for auth/network policy in production.
- **Configuration:** 12-factor via env vars; H2 for local, MySQL for prod; same image across environments.
- **Scalability:** stateless replicas behind a Service and HPA; the harvester is a singleton to keep SOAP load constant regardless of replica count.